TRACE-RPG: 대화형 게임 세계를 위한 감사 가능한 신경-기호 커밋 게이트

익명 저자

***Abstract*—대규모 언어 모델은 표현력 있는 퀘스트와 대화를 생성할 수 있지만, 유창하거나 스키마에 맞는 텍스트만으로 제안 이벤트의 실행 가능성, 권한 적합성, 정식 게임 상태와의 일관성이 성립하지는 않는다. 본 논문은 생성된 이벤트를 신뢰하지 않는 트랜잭션 제안으로 취급하는 기호 커밋 게이트 TRACE-RPG 을 제안한다. 타입 엄격 parser, 외부 행동 정책, 여섯 상태 상대 계열로 묶인 일곱 개의 결정론적 검사가 각 전이를 결정하며, 실패 후보는 최대 $K+1$ 개의 기록된 시도 안에서 제한 수리로 보낼 수 있고 통과 후보는 변경 직전에 다시 검증한다. Validator-error-guided 연산자 ρ 는 직전 후보와 typed validation error 만 읽으며, 이 오류가 유일한 상태 유래 입력이고 권위 있는 `WorldState` 에 직접 접근하지 않는다. 키 없는 SHA-256 콘텐츠 연결, 상태 의미 replay, 할당 완전 계수가 terminal outcome 을 감사 가능하게 만든다. 근거는 네 레인으로 분리한다: 단일 작성 world 의 오프라인 적합성, 단일 모델 라이브 스크리닝, runtime 권한 없는 closed-world 지식 그래프 링크 시뮬레이션, 참여자가 없는 Godot engineering slice. 오프라인 레인은 mechanism 적합성만 확립한다. 라이브 레인은 $K{=}1$ 에서 의도적으로 수리 가능하게 만든 policy-blind regime 에 한해 guided 5/5 대 blind 0/5 commit 을 관찰했으므로 pilot-only 로 남는다. 어느 레인도 모집단 효능, 모델 우월성, 플레이어 경험, 상용 엔진 성능을 확립하지 않는다.**

***Index Terms*—게임 인공지능, 인터랙티브 내러티브, 기호 검증, 제약 생성, 재현성**

I. 서론

인터랙티브 생성은 언어 과제에 그치지 않는다. 후보 이벤트가 자연스러워도 존재하지 않는 아이템을 요구하거나, 화자가 모르는 사실을 주장하거나, 금지된 퀘스트 정보를 공개하거나, 허가되지 않은 효과를 도입하거나, 퀘스트 단계를 후퇴시킬 수 있다. 실행 가능한 텍스트 환경은 그럴듯한 발화와 유효한 전이의 차이를 명시한다 [1], [2]. 근거 기반 대화와 개인화 퀘스트 시스템은 퀘스트, 엔터티, 그래프 맥락의 가치를 추가로 보여주지만 [3], [4], 맥락 자체가 정식 상태 변경을 승인하지는 않는다. **TRACE-RPG 은 생성된 게임 이벤트를 신뢰하지 않는 트랜잭션 제안으로 취급하며, 정식 상태 변경을 외부의 결정론적 commit gate 를 통해서만 허용한다.** 그림 1는 그러한 트랜잭션 하나를 왼쪽에서 오른쪽으로 읽는다.

구조적 생성 제약은 인접한 문제를 해결한다. 문법 제약 디코딩, 언어 모델 질의 언어, 점진 파서는 생성 구조를 제한할 수 있다 [5], [6], [7]. 그러나 구문상 허용되는 이벤트도 현재 세계에 대해서는 거짓일 수 있다. 검색과 메모리는 유용한 근거를 공급할 수 있지만 [8], [9], [10], 검색된 내용 역시 상태 전이의 권한 주체가 아니다.

본 논문은 이러한 관심사를 감사 가능한 트랜잭션 프로토콜로 결합한 구현인 TRACE-RPG 을 제시한다. 현재 기여는 다음과 같다.

1) **C1—계약:** 버전형 신뢰경계 contract 와, proposal/replay 경계에서 unknown key 와 모호한 타입을 거부하는 type-strict parser (IV절; E1);
2) **C2—controller:** 여섯 상태 상대 계열에 걸친 검사 7 개, bounded repair, unchanged fallback, pre-commit revalidation 을 갖춘 validate–repair–commit controller (IV-C절; E1, ENG1);
3) **C3—감사:** SHA-256 연결 record, state-semantic replay, episode-continuity check (IV-D절; E1, ENG1);
4) **C4—harness:** proposal trace 가 존재할 수 없는 경우에도 분류된 terminal row 를 유지하는 assignment-complete 오프라인 적합성 harness (V절; E1);
5) **C5—수 리:** 동 결 repairability class 에 서 blind retry 및 state-reading oracle 과 비 교 한 typed validator-error-guided operator $\rho(a, E)$ (구 현: `counterexample_guided_repair`) (IV-C절; E1, E2).

익명 심사를 위해 제출된 원고입니다.

고정 목표 NPC 운영은 C6 이나 C-RESULT 가 아닌 additive implementation 및 descriptive screening lane 으로 NPC-ENG1/NPC-SCR1 아래 보고한다. 하나의 game-owned Mira goal 을 유지하며, 신경 모델은 typed intent 와 claim 을 제안하고 Python 은 bounded public projection 에서만 계획하며 Godot 이 유일한 canonical writer 다. `player_request` 인 `acquire_object/install_lens` 만 물리적 행동을 일으키고, Mira 의 유일한 NPC world change 는 권한 있는 `tide_marks_hint` 의 `reveal_hint` 이며 `clarify/negotiate` 는 dialogue-only 다. 사전 배정은 NPC-ENG1 16 개 case 와 NPC-SCR1 $12 \times 3 \times 2 = 72$ 개 row(arm 당 24 개), 합계 88 개 (R=40, P=24, O=24) 다. P 는 authenticated Codex typed intent, R 은 작성된 rule interpreter, O 는 independent row 0 인 same-model-family auxiliary reference 다. P/R/O config 는 NPC-SCR1 에만 적용되며 hard ENG1 conformance 와 독립이다. 이는 one-step protocol screening 이며 normal-play full-loop episode 와 분리된 engineering/provenance evidence 이고 efficacy, semantic gold, accuracy ceiling, generalization, human benefit 을 주장하지 않는다. 기계 검증되는 기여 매트릭스에서 생성한 표 I은 C1–C5 를 선행 주제, 근거 레인, 추론 상한에 결합하고, 표 IV는 레인을 정의한다.

레인은 서로 대체할 수 없다 (표 IV). E1 은 단일 작성 world 의 결정론적 적합성, E2 는 고정되지 않은 revision· 제어되지 않은 seed· 구성 state 를 쓰는 셀당 5 회 pilot screening, E3 는 runtime retrieval 없는 closed-world typed-link scoring, ENG1 은 Godot slice 다. NPC-ENG1/NPC-SCR1 은 별도 fixed-goal denominator 와 provenance 를 쓴다. 어느 레인도 population efficacy, model ranking, player benefit, retrieval benefit 또는 C6 를 지지하지 않는다.

TABLE I
기여, 선행 주제, 근거 레인, 추론 상한 (구조화 매트릭스에서 생성)

ID	기여	선행 주제 (참고 문헌 수)	레인	추론 상한
C1	신뢰경계 contract	T1, T2, T3 (17)	E1, ENG1	인코딩 필드 계약 적합성
C2	validate–repair–commit controller	T1, T8 (14)	E1, ENG1	동결 사례의 메커니즘 동작
C3	감사 연결 근거 계층	T4, T7, T8 (6)	E1, ENG1	명명 fixture 의 무결성 · 재생
C4	assignment-complete harness	T1, T4, T6, T7 (20)	E1	설계 사례 정확 집계
C5	반례 유도 연산자 ρ	T1, T8, T9 (8)	E1, E2	동결 class + 단일 regime screen

T1 = 근거 게임 세계; T2 = 검색 · 메모리 에이전트; T3 = 구조 생성; T4 = 환경 · 벤치마크; T5 = 플레이어 경험; T6 = 인간 ·LLM 평가; T7 = 통계 · 재현성; T8 = 계획 · 런타임 개입; T9 = 피드백 수리. 레인은 표 IV에 정의한다. 모든 행은 구현 · 작성 fixture 검증 상태이며 어떤 효능 claim 도 지지하지 않는다.

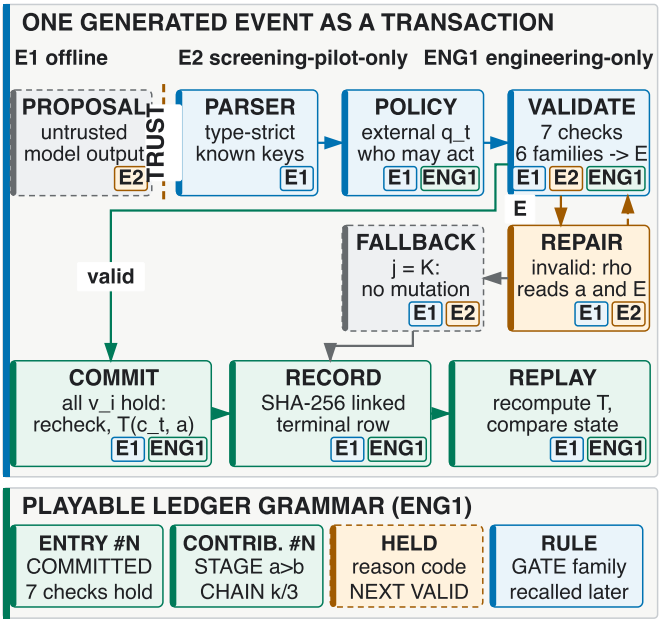


Fig. 1. 생성 이벤트 하나의 트랜잭션 경로다. typed candidate 만 신뢰 경계를 넘고, 외부 policy 와 c_t 를 여섯 상태 상대 계열의 검사 7 개가 확인한다. typed 오류 E 는 bounded ρ(점선) 또는 변경 없는 fallback 으로 이어지고, 통과 후보는 재검사 ·commit·record·replay 된다. 태그는 claim 이 아닌 측정 범위를 표시하며, 아래 strip 은 committed snapshot 에서 파생한 ledger 문법이다.

II. 관련 연구와 연구 공백

A. 인터랙티브 내러티브와 게임 에이전트

KNUDGE 는 퀘스트와 엔터티 명세에 근거한 분기형 NPC 대화를 평가한다 [3]. 지식 그래프 보조 퀘스트 생성 [4], 메모리–성찰–계획 에이전트 [11], 학습 가능한 역할 연기 에이전트 [12] 는 근거성과 개연성 있는 행동의 상호보완적 측면을 다룬다. 플레이어 대상 연구는 선호와 지각된 대화 품질을 평가하고 [13], [14], [15], 2025 년 프리프린트는 역할에 따른 안정성–즉흥성 trade-off 를 보고한다 [16]. 두 흐름 모두 하드 허용 가능성을 선택적 자연스러움 및 플레이어 경험 구성개념과 분리할 동기를 준다.

경험 주도 생성 · 관찰자 표식 참여도 ·vision-language engagement 추론 한계는 future affect work 의 동기지만 [17], [18], [19], affect 는 비권위적이며 본문 구현에 없다.

B. 환경과 벤치마크

TextWorld 와 ScienceWorld 는 명시적인 인터랙티브 상태를 제공하며 [1], [2], MineDojo 는 개방형 지식 집약 embodied 과제를 추가한다 [20]. General Video Game AI 는 에이전트, 게임, 콘텐츠 트랙을 분리한다 [21]. AgentBench 는 이질적인 환경의 에이전트를 연구하고 [22], AgentBoard 는 과정 수준 진단을 추가하며 [23], SOTOPIA 는 사회적 상호작용을 평가하고 [24], BALROG 는 장기 언어 및 시각 게임 추론을 평가한다 [25]. 자동화된 플레이 스타일 에이전트는 상호보완적인 시험 선례이지만 인간 평가를 대체하지 않는다 [26]. 검토한 환경과 benchmark 안에서 transactional authorization, typed repair, semantic replay, assignment-complete accounting 을 하나의 protocol 로 결합한 사례를 식별하지 못했다. 이는 범위가 제한된 문헌 검토 소견이며 최초성 주장이 아니다. NCP-Bench 는 장기 인터랙티브 내러티브의 commitment 와 fact consistency 를 위한 benchmark 를 추가하지만, TRACE-RPG 이나 플레이어 결과를 평가하지 않는다 [27].

C. 제약, 신경-기호 검증, 검색

제약 디코더는 출력 구조를 개선한다 [5], [6], [7]. 반면 TRACE-RPG 은 구조를 독립적인 상태 상대 승인의 입력 조건으로 취급한다. 같은 generate-and-verify 분업은 게임 밖에서 확립되어 있다. LLM-Modulo 프레임워크는 모델이 제안한 계획을 외부의 sound critic 이 수락 또는 거부하게 하고 [28], SayCan 은 언어로 제안된 skill 을 상태 의존 affordance 검사 뒤에만 허용한다 [29]. AgentSpec 은 LLM 에이전트의 런타임 규칙을 trigger–predicate–enforcement 삼중항으로 표현하고 [30], TRACE-RPG 은 술어를 게임 상태에 고정하며 수리 계약과 리플레이 가능한 레코드를 추가한다. Setting the DC 는 D&D 에이전트를 precondition 검사가 있는 tool call 에 근거시키는 반면 [31], TRACE-RPG 은 자유 텍스트 제안을 parsing 과 외부 policy 평가 뒤에만 허용한다. STORY2GAME 은 플레이에 쓰이는 action precondition 과 effect 를 LLM 스스로 생성하게 하지만 [32], 여기서는 policy 가 제안자와 독립적으로 저작된다. TRACE-RPG 은 이 critic 을 typed 반례를 반환하는 결정론적 상태 상대 commit gate 로 구체화한다. 게임 특화 비교 대상은 서로 다른 경계를 보여준다. PANGeA 는 LLM 이 설계자 규칙에 따라 콘텐츠를 판정하고 수정하도록 해 검증하고 [33], IVIE 는 기호 검증으로 인터랙티브 픽션 세계를 점진적으로 생성하며 [34], 같은 계보의 2026 년 후속 시스템은 LLM 을 저작된 world-state transformation 선택으로 제한해 [35] 구성상 일관성을 얻는다. 이 논문의 트랜잭션, 감사 레코드, 수리 연산자 질문은 별도의 근거 대상이다. 어느 비교도 우월성을 뜻하지 않으며, 유효성은 각 시스템이 인코딩한 술어에 한정된다. JSONSchemaBench 는 구조적 constrained-output compliance, coverage, quality 를 측정하지만, 이는 상태 상대 semantic authorization 을 확립하지 않는다 [36].

G-Retriever, HippoRAG, Mem0 는 subgraph retrieval 과 장기 메모리를, Generative Agents 와 Character-LLM 은 행동 memory 와 persona 를 다룬다 [8], [9], [10], [11], [12]. TRACE-RPG 은 이런 context 를 받을 수 있지만 commit authority 를 주지 않으며 효익은 측정하지 않았다.

D. 감사 가능성과 재현성

과정 지표, sparse-run caution, reproducibility 와 environmental reporting 은 failure retention 과 frozen artifact 를

뒷받침한다 [23], [37], [38], [39]. Human/LLM 평가와 향후 participant 분석에는 construct/rater/bias control, justified margin 과 random-effects 구조가 필요하다 [40], [41], [42], [43], [44], [45]. Record-and-replay 는 경험 재사용을 지원하지만 [46], TRACE-RPG 은 admitted transition 의 semantic replay 만 주장한다. 벤치마크 오염 완화 연구는 question-level leakage check 의 필요성을 뒷받침하지만, 동결된 denominator 를 변경하거나 game-agent efficacy 결과를 만들지는 않는다 [47].

제한된 novelty 는 구성요소가 아니라 구현된 결합이다. 선행 연구는 runtime enforcement [30], symbolic game-state management [31], [32], feedback repair [48], record/replay [46], action interposition [49], classical preconditions/effects [50], permitted-transition enforcement [51], transaction abort [52], authored social logic [53], belief-aware narrative planning [54] 를 각각 다룬다. TRACE-RPG 은 이를 game-state commit boundary 에서 결합한다. C1–C2 는 independent contract 와 validate–repair–commit 을, C3–C4 는 checksum-linked/state-semantic replay·episode continuity·assignment-complete accounting 을, C5 는 typed-error repair 를 구현한다. 보고서에 기능이 없다고 해서 구현 전체에 없다고 추론하지 않는다.

III. 문제 정의와 보장 경계

설명을 위해 시점 t 에서 validation 이 사용하는 controller 입력을 다음과 같이 분해한다.

$$c_t = (G_t, q_t), \tag{1}$$

여기서 G_t 는 typed snapshot field 를, q_t 는 해당 snapshot 과 함께 저장되는 외부 작성 action-policy, disclosure, quest-stage 제약을 나타낸다. 종단 record history $h_{<t}$ 는 `WorldState` snapshot 과 별도로 유지된다. Validation predicate 는 이를 읽지 않으며 구현은 log 에서 상태를 재구성하지 않는다. 이 one-step model 에서 수락된 전이는 q_t 아래 G_t 를 갱신하고 q_t 를 그대로 다음 단계로 전달한다. 정책 버전 교체는 이 전이의 범위 밖이다. 검색, 내러티브 점수, 모델 rationale, 메모리 요약, 미래의 감정 추정은 비권위적 proposal context 이다.

신뢰하지 않는 proposer 가 후보 a_t 를 출력한다. 타입 엄격 parsing 은 알 수 없는 top-level field 의 거부를 포함한 공유 candidate-key contract 를 먼저 강제하고 선언 스키마를 확립한다. validator 는 다음을 반환한다.

$$V(c_t, a_t) = (v_{\text{policy}}, v_{\text{pre}}, v_{\text{reach}}, v_{\text{know}}, v_{\text{disc}}, v_{\text{elig}}, v_{\text{stage}}, E_t), \tag{2}$$

여기서 E_t 는 구조화 오류 집합이다. 상태 변경은 다음과 같다.

$$c_{t+1} = \begin{cases} T(c_t, a_t), & \bigwedge_{i=1}^{7} v_i = 1, \\ c_t, & \text{그 외.} \end{cases} \tag{3}$$

Controller 는 인코딩된 계약 아래 다음 구현 불변량을 주장한다. 파일럿은 이를 일반 정리로 제시하지 않고 deterministic valid, invalid, repair, fallback, replay, fault-injection fixture 로 해당 mechanism 을 시험한다. **I1 (소진 실패 불변성):** 예산 K 까지 반환된 어떤 후보도 검증을 통과하지 못하거나 proposal/controller 실행이 commit 전에 실패하면 반환 상태는 공급된 prior state 와 같다. **I2 (제한된 기록 시도):** 모든 proposal 과 repair callback 이 반환한다는 조건에서 repair budget $K \geq 0$ 은 최대 $K+1$ 개의 candidate attempt 를 기록한다. 성공적인 commit 은 변경 직전 수락 후보에 대한 방어적 validation 을 한 번 추가한다. Runtime 은 callback deadline 을 강제하지 않으므로 I2 는 wall-clock 종료 보장이 아니다. **I3 (효과 권한):** 수락 후보는 `ActionPolicy.allowed_effects` 밖의 선언된 fact effect 를 `CandidateAction.effects` 에 포함하지 않으며, quest-stage target 도 해당 policy 에 의해 명시적으로 승인된다. **I4 (상태 의미적 리플레이):** 동결된 스키마와 결정론적 전이 아래 수락된 trace 는 명시된 terminal state 를 재구성한다.

이 주장된 불변량은 올바른 정책 작성, 선언된 후보 필드로의 완전하고 충실한 추출, 결정론적 소프트웨어 버전, callback 반환, 단일 프로세스 파일 소유권을 가정한다. 상태 의미적 리플레이는 기록된 구조화 후보와 전이를 재검증하지만, 중간 수리의 provenance, reasoning, prompt, 확률적 생성은 재현하거나 검증하지 않는다. 구조화 필드에서 누락된 자연어 함의는 보장 범위 밖이다.

IV. TRACE-RPG 아키텍처와 알고리즘

A. 신뢰 경계와 계약

그림 1는 모델 또는 recorded adapter 를 canonical owner 밖에 둔다. Parser 는 알려진 필드의 모호한 숫자 또는 collection type 을 거부하고 공유 key contract 에 따라 알 수 없는 top-level candidate field 도 거부한다. action policy 는 candidate 와 별도로 공급되며 required precondition 과 allowed effect 를 정의한다. Game bridge 는 observation, proposal, validation, rejection, fallback, commit 을 전달할 수 있는 versioned event interface 이며, 정식 상태 변경은 유효한 commit 만 승인한다. 이는 interface 를 분리하지만 상용 엔진으로의 portability 를 아직 입증하지 않는다. 근거는 unknown-key rejection parity 에 한정되며 proposal default 와 replay parsing 의 동등성을 확립하지 않는다. 표 II는 구현 검사 7 개를 여섯 의미 계열로 묶는다. Quest eligibility 와 quest-stage monotonicity 는 별도 검사로 유지된다. 이 인코딩 경계는 의미적 완결성 주장이 아니다.

B. 고정 목표 *NPC* 통합

Mira 의 고정 목표는 항구 신호 복구 뒤 허가된 조석 단서를 제공하는 것이다. 각 turn 은 typed intent/claim proposal 에서 시작해 proposal → verification → public symbolic plan → player/Mira role gate → actual result → fresh observation-only replan 으로 진행된다. Verification 은 공개 flag 에서 claim 상태를 판정하고, Python 은 omniscient state 가 아닌 공개 Boolean projection 에서 bounded BFS 를 수행해 첫 유효 step 만 내보낸다. `acquire_object`/`install_lens` 는 player request 로만 수행되고 Mira 는 알려진 `tide_marks_hint` 와 disclosure authorization 이 있을 때만 `reveal_hint` 를 commit 한다. `clarify`/`negotiate` 는 canonical effect 가 없는 dialogue 이며 경로가 없으면 `NO_ROUTE` 와 물리적 step 없음으로 끝난다. 결과 뒤에는 fresh observation/state reference 와 직전 typed intent 만 재사용하고 utterance/claim 은 재해석하지 않는다.

각 요청은 wire 의 `npc-goal-recovery/1.1.0` 계약을 따른다. `config_hash` 는 `configs/npc-goal-recovery.json` raw bytes 의 SHA-256 이며 논문에는 prefix `d63f5679f360d3fd` 만 싣고 전체 digest 는 `research/academic-pipeline/npc-goal-recovery/verified-run-002/` 의 validated packet hash manifest 가 보존한다. `policy_hash` 는 prefix `af0a1075294b1163`, `project.godot` source hash 는 prefix `2e7966bf2ac6` 로 같은 manifest 에 기록된다. utterance event 의 `raw_utterance_sha256` 는 UTF-8 바이트열의 SHA-256

TABLE II
인코딩된 Validator 경계

술어	거부 대상	실패 시 상태
Action policy	알 수 없는 action/type	변경 없음
Precondition	누락/거짓 requirement	변경 없음
Reachability	접근 불가 object	변경 없음
NPC knowledge	선언되지 않은 known fact	변경 없음
Disclosure	forbidden fact	변경 없음
Quest eligibility	부적격 quest action	변경 없음
Quest stage	후퇴 또는 비인가 stage	변경 없음

1: $a \leftarrow \text{Propose}(c)$; $j \leftarrow 0$
2: **repeat**
3: $(ok, E) \leftarrow V(c, a)$; Record$(a, E)$
4: **if** ok **then**
5: **assert** $V(c, a).ok$; **return** Commit(c, a)
6: **if** $j = K$ **then return** Fallback(c)
7: $a \leftarrow \text{Repair}(c, a, E)$; $j \leftarrow j + 1$
8: **until terminal**

Fig. 2. Validate–repair–commit 절차. 최대 $K + 1$ 개의 candidate attempt 를 기록하며, 성공 경로는 상태 변경 전 방어적 validation 을 수행한다.

commitment 이고 observation event 에서는 `utterance` 와 digest 를 모두 `null` 로 둔다. loopback 은 `127.0.0.1:8768`, browser origin 은 `localhost` 및 `127.0.0.1:8877` 이며, Codex 는 비권위적 OAuth CLI interpreter 이고 P 는 rule fallback 으로 대체하지 않는다. 고정 identifier 는 `harbor-signal-restored-authorized-tide-lead-v1` (goal version 1, NPC `captain_mira`) 이며 utterance/claim/model output 은 이를 바꾸지 못한다. episode total/deadline 과 intent call/action attempt/replan/BFS expansion 예산은 누적된다. 적용된 acquire/install/reveal 은 revision 을 정확히 1 증가시키고 hash 를 바꾸며 dialogue 와 미적용 결과는 state reference 를 유지하고 `STALE` 은 채택하지 않는다. Redacted recorded-intent replay 는 모델 호출 0 회이고 native replay 경로는 `--npc-experiment/engine_fixture_reexecution` 이며 `--npc-replay` 는 사용하지 않는다. Live cancellation 은 local invalidation 일 뿐이며 `Core.cancel` no-publication 은 unit/controller 속성이지 provider 완료 취소 보장이 아니다.

C. Validate–Repair–Commit

그림 2은 제어기의 추가 패키지 없는 의사코드를 제시한다. "Record" 는 in-memory trace 표현에 추가하는 것을 뜻하며, 영속 쓰기는 종단 일관성 검사 뒤에만 수행한다.

검증 오류는 구조화된 error code 로 repair callback 에 노출된다. repair 는 상태를 직접 변경하지 않으며, 예산 소진 시 변경 없는 결정론적 fallback 을 반환한다. 그림 1이 종단 경로를 나타낸다. 파일럿은 구현된 두 callback 을 비교한다. *typed validator-error-guided* 연산자 (구현: `counterexample_guided_repair`) 는 직전 후보 a 와 typed validator 오류 E 만을 소비하며, 이 오류가 유일한 상태 유래 입력이고 권위 있는 `WorldState` 에는 직접 접근하지 않는다.

$$\rho(a, E) = a \oplus \bigcup_{e \in E} \delta(e), \tag{4}$$

여기서 각 $\delta(e)$ 는 표 III의 오류 class 별 결정론적 edit 이고 $\oplus$ 는 그 field edit 들을 적용한다. 각 edit 은 오류 entity 당 최대 하나의 candidate field 만 수정하므로 시도당 변경 field 수는 $|E|$ 로 유계이고, 동일한 오류 집합 E 를 두 번 적용할 때에만 고정점이 되며 오류 집합이나 시도가 바뀌는 과정의 수렴을 뜻하지 않는다. 같은 field 에서 동시에 요구되고 거부되는 entity 는 수리 불가로 선언되어 수정되지 않는다. 상태를 읽는 참조 *callback* 은 반대로 오류 집합을 폐기하고 정책이 규율하는 field(precondition, effect, 두 quest-stage field) 를 권위 있는 상태에서 직접 재구성한다. 이 callback 은 선언 무결성 field(required object, used fact, disclosed fact) 를 의도적으로 수정하지 않는다. 의존성이나 disclosure 의 선언을 지우는 것은 후보를 수리하는 것이 아니라 위반을 게이트 뒤로 세탁하는 것이기 때문이다. 이는 상태 판독 가능 수리에 대한 oracle 상한이며, 그 결과를 배포 가능한 수리 방법의 근거로 읽어서는 안 된다.

TABLE III
유도 연산자 ρ 의 오류 class 별 edit

Validator 오류 class	후보에 대한 edit $\delta(e)$
정책 precondition 누락	명명된 predicate 삽입
정책 effect 누락	명명된 effect 삽입
정책 effect 위반	명명된 effect 제거
불충족 precondition	명명된 predicate 제거
비인가 stage 변이	`quest_stage_effect` 초기화
알 수 없는 action type	no-op (등록된 대안 없음)
도달 불가 object; knowledge, disclosure, stage class	no-op (후보 국소 edit 은 세탁이거나 상태 지식 필요)

D. 무결성, 의미적 리플레이, 계수

각 result row 는 키 없는 SHA-256 무결성 체크섬을 가지며, proposal outcome 은 추가로 trace checksum 과 연결된다. 이 hash 들은 우발적이거나 시험 대상인 콘텐츠 변경을 탐지하지만 signature, message-authentication code, writer identity 가 아니며 콘텐츠와 checksum 을 모두 다시 쓸 수 있는 행위자에 대한 보호도 아니다. 의미적 리플레이는 prior state 와 기록된 모든 candidate/validation pair 를 엄격히 parsing 하고, final 이 아닌 모든 attempt 가 invalid 임을 요구하며, terminal transition 을 재계산하고, 명시된 final state 와 비교함으로써 byte 무결성을 넘어선다. Episode replay 는 인접 JSONL record 사이의 상태 연속성도 요구한다. Replay 는 candidate $j + 1$ 을 만든 repair-generation 단계를 인증하거나 다시 실행하지 않는다.

Terminal record class 는 commit, symbolic fallback, adapter failure, controller failure 이다. Controller-failure row 는 동결된 assignment denominator 에 남고 상태를 변경하지 않지만, candidate outcome 이 생성되지 않았다면 완성된 proposal trace 가 없어도 된다. 따라서 result completeness 가 모든 terminal row 의 trace 존재를 뜻하지 않는다. 실행 전 assignment key 는 $(\text{run}, \text{arm}, \text{scenario}, \text{seed}, \text{model}, \text{revision}, h_{\text{controller}}, h_{\text{input}}, h_{\text{prior}})$ 이다. 집계는 중복, 누락, 예기치 않은 key 를 거부하고 symbolic, adapter, controller failure 를 각각 보고한다.

V. 실험 레인과 근거 경계

표 IV는 어떤 count 를 읽기 전에 각 레인의 설계, 단위, 비교, 상한을 고정한다.

A. *E1* 오프라인 적합성 설계

파일럿은 다음을 묻는다. (PQ1) 유효 후보는 commit 되고 명명된 invalid-policy 사례는 변경 없이 유지되는가? (PQ2) 동

TABLE IV
실험 레인 요약: 설계, 단위, 비교, 추론 상한 (실험 매트릭스에서 생성)

레인	설계	단위 / 예산	비교	상한
E1	오프라인 적합성: 작성 · 동결 fixture, 단일 world	gate 13; arm 당 repair 12; fault 10; adapter 7; guard 3	reject / blind / ρ / oracle	메커니즘 적합성
E2	RQ2 screening-pilot-only: hosted proposer 1 개, matched candidate	5 cell × 5 call; $K=1$	regime 별 ρ / blind	screening-pilot-only 전이 screen
E3	KG/온톨로지 시뮬레이션: closed-world typed-link holdout	node 43; 참조 edge 106; typed 24; score 210	degree baseline / 고정 전략 6 종	구성 결과만
ENG1	Engineering-only Godot/Web: 엔진 로컬 policy mirror, 참여자 없음	fixture 4; check 52; smoke 8; archetype 5	작성 fixture 경로	엔진 로컬 engineering-only 적합성

E1–E3 와 ENG1 은 서로 대체할 수 없으며 분모가 다르다. E2 는 screening-pilot-only 이고 ENG1 은 engineering-only 다. 어떤 레인도 모집단 효능, 모델 순위, 플레이어 경험, 런타임 검색 효익, 상용 엔진 성능을 확립하지 않는다.

결된 모든 repairability class 의 작성 사례에서 rejection-only, 블라인드 동일 후보 retry, typed validator-error-guided 연산자 ρ, 상태를 읽는 참조 callback 이 예상 결과를 만드는가? (PQ3) 동결된 checksum, state-semantic, linkage, continuity mutation 이 지정된 검사 연산에서 거부되고, 두 번째 파일 append 의 주입된 실패가 예상 pair-write rollback 검사를 작동시키는가? (PQ4) 엄격한 adapter 와 assignment manifest 가 할당 사례마다 정확히 하나의 분류된 terminal row 를 유지하거나 fail closed 하는가? (PQ5) proposal 및 replay parser 가 그 외에는 완전하고 유효한 candidate 에 unknown top-level key 하나를 추가한 사례를 모두 거부하는가?

사례는 의도적으로 설계된 적합성 픽스처이며 모델 또는 게임 모집단의 표본이 아니다. 네 도메인은 validator/state isolation, repair-arm 비교, record/trace fault injection, adapter/accounting contract 이다. 모든 repair fixture 는 실행 전에 동결된 repairability class 를 선언한다. *guided-repairable*(구조화 오류 payload 만으로 ρ 에 충분한 정보가 담김), *oracle-only*(수리에 권위 있는 상태 판독이 추가로 필요함), *irreparable*(구현된 어떤 arm 도 commit 할 수 없음) 이며, 13 개 gate fixture 는 유효 control 하나와 고립된 음성 12 개로 구성되고 12 개 repair case 는 별도 arm denominator 를 이룬다. 각 class 별 arm 기대 결과는 fixture 계약의 일부이며, 모든 기대값은 실행 전에 명시한다.

측정치는 정확한 count 이다. expected/observed validator-class agreement, rejected 또는 controller-failure 사례의 state mutation, repair arm 과 repairability 별 commit 및 attempt 결과, 지정된 연산에서 거부된/사전 명세된 detectable fault, replayed/committed trace, rejected/injected manifest violation, 사전 명세한 unknown-key 음성 회귀에 대한 proposal/replay 거부 일치를 센다. 마지막 측정은 결정론적 parser-contract parity 이며 실증적 safety outcome 이 아니다. Exception class 만으로 안정적인 detector layer 를 식별할 수 없으므로 detector 귀속 결과는 보고하지 않는다. 의도적으로 설계된 픽스처에 p-value 나 모집단 confidence interval 을 부여하지 않는다. Adapter 사례가 지닌 latency 및 token 상수는 입력 스키마가 JSON-Schema `const` 로 고정한 합성값이다. 이 값들은 계수 필드 전파만 확인하며 provider, runtime, device 에 대한 측정값이 아니다. 동결된 오프라인 packet 은 fixture, assignment, schema, trace 를 포함한 result, 코드 revision, checksum 을 결합하며, 보고는 재현성과 측정 경계 지침을 따른다 [38], [39].

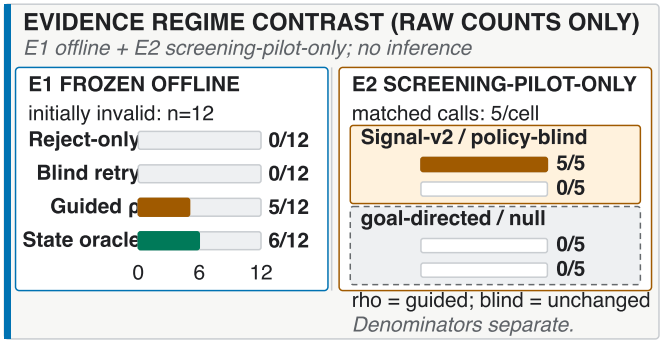


Fig. 3. E1 동결 오프라인 count 와 E2 screening-pilot-only matched count 를 사용한 evidence-regime 대비. E1 은 처음 invalid 로 설계된 12 개 사례에서 네 arm 의 commit/case 를 보여 준다 (0/12, 0/12, 5/12, 6/12). E2 는 Signal-v2/policy-blind 대비 (ρ 5/5 대 blind 0/5) 를 goal-directed/goal-blind null regime (0/5 대 0/5) 와 분리한다. Raw count 는 기술적이며 denominator 는 통합되지 않으므로 효능, 모집단, 모델 순위 또는 플레이어 경험에 대한 추론을 지지하지 않는다.

B. *E2* 라이브 *RQ2* 스크리닝

라이브 확장은 블라인드 동일 후보 retry 와 ρ 를 동일한 $K=1$ 에서 비교한다. 각 셀에서 seed 로 구분한 hosted proposer `gpt-5.6-sol` 호출 5 회가 후보 하나씩을 만들고, 두 arm 은 같은 최초 후보를 받아 호출 내 제안 차이를 제거한다. 조건은 policy 공개 여부와 자명한 효과 0 경로의 존재를 바꾼다. 동결된 packet 은 현재 셀과 폐기된 v1 진단을 함께 보존하고 모든 영수증을 SHA-256 으로 결합하며 pilot-only 로 표시된다. Seed label 은 hosted sampler 를 제어하지 않고, hosted revision 은 고정되지 않았으며, token 회계는 없고, 구성 상태는 모집단 표본이 아니다. 따라서 p-value, confidence interval, 모델 순위 없이 정확한 기술 count 와 실패 class 만 보고한다.

C. *E3* *KG*/온톨로지 링크 제안 시뮬레이션

세 번째 레인은 권한을 가진 `WorldState` 와 분리된 engineering-only closed-world 시뮬레이션이다. graph node 43 개와 reference edge 106 개, 검토된 typed edge 24 개가 type/domain/range 검사와 competency question 6 개를 통과하며, 작성형 질의 6 개를 고정 전략 7 개로 채점했다. Retained typed-lexical 전략 (S2) 은 작성된 holdout 6/6 을 복원했고 ($P = R = F_1 = 1.000$, realistic-tie MRR $= 0.944$, Brier $= 0.131$, Sem@3 $= 1.000$), degree baseline 은 하나도 복원하지 못했다. 이는 구성 결과이며 OWL/SHACL 적합성, 의미 완전성, runtime retrieval, 어떤 효능 주장의 근거도 아니다.

VI. 파일럿 결과

A. 구조 필드 게이트 적합성

파일럿 loader 는 구현된 validator code 를 정확히 한 번씩 고립시키는 fixture 집합과 유효 control 1 개만을 허용한다. 따라서 13/13 일치와 구현된 오류 코드 12 종의 관찰은 측정된 성공률이 아니라 하니스의 구성 불변량이다. 이 실행이 보여주는 것은 loader 가 강제하지 않는 부분, 즉 고립된 음성 fixture 12 개가 각각 다른 코드가 아니라 작성된 기대 코드와 일치했고 비commit 경로가 전체 정준 상태를 유지했다는 관찰이다. 이는 단일 world state 에서 저자가 설계한 구조 필드 oracle 에 대한 구현 적합성이며 독립 의미 판정의 정확도가 아니다.

B. 설계 픽스처에서의 수리 *arm* 비교

처음부터 유효하지 않은 설계 case 12 개는 guided-repairable 5 개, oracle-only 1 개, irreparable 6 개의 동결된 repairability class 로 분할된다. Rejection-only 와 블라인드 동일 후보 retry 는 각각 0/12 commit 이었다. validator-error-guided 연산자 ρ 는 guided-repairable case 5/5 를 commit 했고, 설계상 oracle-only 0/1, irreparable 0/6 였다. 모든 ρ commit 은 candidate field 를 평균 1.0 개 수정했으며, ρ 는 직전 candidate 와 typed validator error E 만 읽고 error 가 state-derived input 의 유일한 경로이며, 실패한 모든 arm 실행은 정준 상태를 변경 없이 유지했다. 상태를 읽는 참조 callback 은 guided-repairable 5/5 와 oracle-only 1/1 를 commit 했고 irreparable 은 0/6 였다. 이 정확한 count 는 세 메커니즘을 설계 픽스처 위에서 분리한다. 블라인드 retry 는 validator error 를 commit 으로 전환하지 못하고, ρ 는 오류 payload 에 충분한 정보가 담긴 case 만 전환하며, oracle 은 상태 지식이 필요한 case 를 추가로 전환한다. commit 된 guided case 5 건이 처음 받은 validator code 는 각각 `POLICY_PRECONDITION_OMISSION+UNSATISFIED_PRECONDITION`, `POLICY_EFFECT_OMISSION`, `POLICY_EFFECT_VIOLATION`, `UNSATISFIED_PRECONDITION`, `POLICY_QUEST_STAGE_EFFECT_VIOLATION` 이며, 각 case 는 표 III의 해당 edit $\delta(e)$ 만으로 유효해졌다. 이는 동결 픽스처에 대한 연산자 적합성이며, 어떤 live model 의 수리 품질도 아니다.

C. 무결성, 경계, 배정 회계

현재 명세가 검출 가능하다고 지정한 10 개 fault fixture 는 모두 지정된 검사 연산에서 거부됐다 (10/10). 이 파일럿은 안정적인 typed detector code 까지 대조하지 않으므로 detector layer 귀속을 입증하지 않는다. 별도로, 동일하게 기록된 유효 수리 앞의 무효 선행 후보를 다른 무효 후보로 치환하고 체크섬을 재계산한 알려진 provenance 경계는 예상대로 재생을 통과했다 (1/1 미검출). 따라서 키 없는 체크섬을 재계산할 수 있는 작성자를 방어하지 않으며, 재생은 repair 생성 연산의 출처를 인증하지 않는다. 열린 경계 sentinel 두 건—자연어 disclosure 미추출과 required object 의 후보 · 정책 동시 누락—은 2/2 가 의도대로 인코딩상 허용됐고 safety pass 는 0 건이었다. 과거 unknown top-level field 허용 경계는 이후 닫힌 음성 회귀로 재분류되었으며, 완전한 12-field candidate 에 unknown key 하나를 추가한 fixture 를 proposal·replay parser 모두 구체적으로 거부했다 (1/1). 이는 parser rejection parity 이지 의미 안전성의 증거가 아니다. Adapter/accounting 7 개 배정에서는 commit 1, 기호 fallback 1, adapter failure 5 였다. 합성 telemetry 필드는 2/7 배정에서 채워져 전파됐으며, 이는 측정값이 아니라 회계 필드 전파 확인이다. 배정 guard 3/3 가 중복 · 누락을 거부했다. 이 수치는 모두 합성 offline fixture 의 raw count 다.

표 V와 표 VI은 정확한 설계 픽스처 count 를 보고한다. 13/13 gate 일치는 loader 구성 불변량이고, 각 repair arm 은 고유 case 12 개 ($5+1+6$) 를 가지며, 라이브 5 개 셀의 25 개 제안 호출은 오프라인 denominator 밖에 있다. 수리 표는 동결된 repairability class 별로 네 arm 을 분리하며, 어느 표도 모집단 또는 live-model 추정을 지지하지 않는다.

D. 라이브 스크리닝 파일럿 *(pilot-only)*

별도 스크리닝 파일럿은 hosted proposer gpt-5.6-sol 을 호출했다. 셀마다 seed 로 구분한 제안 호출 5 회와 수리 예산 $K=1$ 을 사용했고, 각 호출의 최초 후보 하나를 블라인드 동일 후보 retry 와 ρ 가 공유했다. Hosted revision 은 고정되지 않았고 token 회계는 제공되지 않았으며 seed 는 hosted sampler 를 제어하지 않으므로 이 호출들은 독립 무작위 표본이 아니라 준반복이다. 따라서 추론 통계나 모델 순위를 보고하지 않는다.

표 VII에서 유도 우위는 의도적으로 guided-repairable 오류가 나오도록 구성한 signal-repair state(Signal-v2) 의 policy-blind 셀에서만 나타났다. 최초 후보 5/5 가 모두 무효였고 ρ 는 5/5, 블라인드는 0/5 를 commit 했다. 나머지 현재 셀 4 개는 유도 우위를 보이지 않았다. 3 개 셀은 최초 후보가 이미 유효했고, 동결 기저의 goal-directed 셀은 QSR 5/5 인 null regime 로 양 arm commit 이 0/5, 0/5 였다. 현재 셀의 모든 noncommit arm 결과 15/15 가 prior-state hash 를 보존했다. 이 결과는 해당 오류 regime 에서의 mechanism transfer 만 지지하며 모집단 효능, 모델 승격, 표본 효율 일반화는 지지하지 않는다. 따라서 두 스크리닝 관찰은 pilot-only 이며 확증적 전이 claim 은 열린 채로 남는다. 폐기된 v1 진단에서는 동반 오류가 최대 3 개에서 수리 불가 knowledge 오류 1 개로 줄었지만 commit 되지 않아, 오류 하나라도 수리 불가이면 부분 수리가 합성되지 않음을 보여 주었다.

a) NPC 결과 생성 경계.: NPC-ENG1 은 16 개 deterministic native R rule-control hard conformance, NPC-SCR1 은 validated-v5 의 12 개 case $\times$ 2 회 $\times$ 3 arm(P/R/O) screening 이다. 모든 assigned row 와 pre-trigger/parse/deadline-cancel/duplicate-stale/controller/no-route failure 를 분모에 보존한다. P 는 authenticated Codex typed-intent interpreter, R 은 작성된 rule interpreter, O 는 same-model-family auxiliary reference 다. P 24 개는 사전 배정 분모일 뿐이며 관측 count 와

TABLE V
E1 오프라인 적합성: 수리 가능성 class 별 정확 count (설계 fixture)

Arm	정보원	Commit / 사례			평균 수정 [e]
		G-rep.[f]	O-only[f]	Irrep.[f]	필드 수
Rejection-only (K=0)	없음	0/5	0/1	0/6	—
블라인드 retry	없음	0/5	0/1	0/6	—
Validator-error-guided ρ	오류 집합 E	5/5	0/1	0/6	1.0
상태 판독 oracle	권위 상태 + 정책	5/5	1/1	0/6	1.0

[e]Commit 된 수리의 변경 candidate field 평균 개수 (최소성 proxy); 실패한 arm 실행은 상태를 변경하지 않았다. E1 은 처음부터 invalid 한 12 개 설계 case 를 사용하며 rejection-only/blind 는 0/12, 0/12, validator-error-guided ρ 는 5/12, state-reading oracle 은 6/12 commit 이다. [f]Commit/사례 수. G-rep. = guided-repairable, O-only = oracle-only, Irrep. = irreparable(동결 repairability class); 모집단 효능이나 live-model 품질을 뜻하지 않는다.

TABLE VI
E1 오프라인 적합성: 동결 파일럿 검사와 종단 결과 (행 간 직접 비교 불가)

검사 또는 종단군	정확 결과	해석 범위
게이트 fixture 일치	13/13	저자 oracle 일치
알려진 경계 허용	2/2	인코딩 한계, safety pass 아님
닫힌 unknown-key 거부	1/1	proposal/replay parser parity
탐지 가능 무결성 결함 거부	10/10	지정된 검사 연산
알려진 provenance 경계 허용	1/1	repair 생성 출처 미인증
Adapter 종단 class	1 commit; 1 fallback; 5 실패 / 배정 7	배정 7 건의 분류 완전성
Manifest guard 거부	3/3	주입 결함 fail-closed

행마다 분모와 성공 방향이 다르므로 비율처럼 순위를 비교하지 않는다. 모든 값은 설계 픽스처의 정확한 감사 사실이다.

TABLE VII
E2 screening-pilot-only: 라이브 스크리닝 정확 집계 (5 개 cell, 셀당 제안 5 회, K=1)

기저 / 조건	최초 유효	ρ vs blind commit	Δ	최초 오류
동결 / 공개	5/5	5/5 vs 5/5	+0	—
동결 / 비공개	5/5	5/5 vs 5/5	+0	—
동결 / 목표-비공개	0/5	0/5 vs 0/5	+0	QSR
Signal-v2 / 공개	5/5	5/5 vs 5/5	+0	—
Signal-v2 / 비공개	0/5	**5/5 vs 0/5**	**+5**	PEO+PEV+PPO

QSR = quest-stage regression, PEO = policy-effect omission, PEV = policy-effect violation, PPO = policy-precondition omission. Δ 는 동일 후보 (matched candidate) 에서 ρ commit 수 minus blind commit 수다. 5 개 cell × cell 당 5 회 호출이며, 3 개 cell 은 최초 5/5 유효, goal-blind cell 은 QSR 5/5 로 양 arm 0/5, 0/5 commit 인 null regime 이다. Signal-v2 blind 는 ρ 5/5 대 blind 0/5 로 분리되었다. 모든 수치는 screening-pilot-only 이며 추론 통계 · 모델 순위가 아니다.

efficacy 는 validated packet 뒤 generator 가 실제 값으로만 반영한다. Native Codex full-loop demo 는 case pool 밖 engineering artifact 다. ENG-X01, SCR-X01, D02, QA-001, C-RESULT, Frozen8, Frozen16 및 v1–v4 provenance pointer 는 source artifact 에 보존한다. 검증된 `verified-run-002` packet 에서만 generator 가 `npc_results_ko.tex/npc_tables_ko.tex` 를 갱신한다.

E. 고정 목표 *NPC* 운영 *(NPC-ENG1/NPC-SCR1)*

Mira 의 고정 목표는 항구 신호 복구 후 허가된 조석 단서를 제공하는 것이다. Python 은 typed intent 와 공개 관측으로 bounded BFS 계획을 계산한다. 획득 · 설치는 플레이어가, 허가된 `reveal_hint` 는 NPC 가 수행한다. Godot 은 session/epoch/request/goal/policy/revision/hash 를 동기 검사한 뒤 기존 writer 를 await 없이 호출한다. 결과와 새 관측이 다음 계획을 결정하므로 단일 후보의 validator-error repair 와 구별된다.

NPC-ENG1 은 실제 엔진 snapshot 에서 시작한 R arm 의 결정론적 engineering 사례 16 개 (assigned_failure=2, completed=14) 다. NPC-SCR1 은 P/R/O 각 24 개, 전체 72 개 배정과 실패를 모두 보존한다 (assigned_failure=12, completed=60). P 는 인증된 Codex CLI 실행 시도 24 회, 반환 decision 24 개, 배정 실패 4 개를 기록했다. 이는 원격 과금 호출 횟수가 아니다. 요청 모델: gpt-5.6-sol; decision 에 기록된 모델 식별자: gpt-5.6-sol. 후자는 설정된 selector 이며 provider revision 이 아니다. provider revision 은 사용할 수 없고 null 식별자는 미상으로 남긴다. 전역 설정은 모델 식별 근거가 아니다. 실제 모델 계열 일치는 unverified(검증되지 않음) 이다. R 은 작성된 규칙을 사용한다. O 의 `same_model_family_auxiliary` 는 protocol source classification 일 뿐이고 non-independent 이며 실제 동일 모델을 입증하지 않는다. 독립 원본은 0 개이고 O 는 oracle 이나 semantic gold 가 아니다. 표 VIII과 그림 4은 bounded protocol-conformance 고정 사례의 기술적 계수이지 회복 효능 · 우월성 · 일반화 추정이 아니다. efficacy 는 pending 이며 토큰 · 비용은 미측정이다. 읽기 전용 CLI 와 격리 작업 디렉터리를 사용했지만 tool-free 추론이나 적대적 기밀성은 검증하지 않았다. 검증된 사전 작성 stale-control ACK 거부는 arm 별 P=4, R=4, O=4 건 (SCR-T01 (stale_revision), SCR-T02 (stale_policy)) 이며, 12 개 인스턴스 모두 대응하는 음성 대조 gate 가 모두 통과했다. 따라서 이 행들은 assigned failure 로 분모에 남기되 method-performance 결과로 해석하지 않으며, 기타 assigned failure 0 개는 미분류로 남긴다.

TABLE VIII
NPC-SCR1 고정 사례 실행 계수. NPC-ENG1 의 16 개 사례와 분리하며 모든 실패를 분모에 보존한다.

Arm	배정	완료/실패	목표 도달	회복 E/U
P	24	20/4	4	24/0
R	24	20/4	4	24/0
O	24	20/4	4	24/0

E: 회복 평가 가능, U: 도달 불가; NA: 미측정. 목표 관측 계수는 회복률이 아니다. Replay 적합성은 실패 행을 완료로 바꾸지 않는다. O 의 독립 원본은 0 개다.

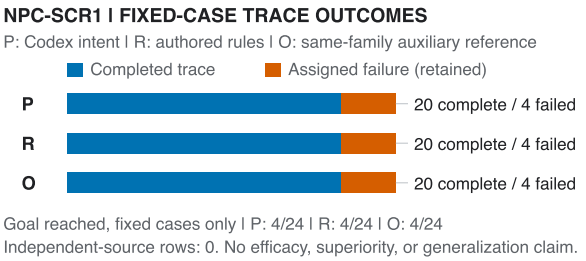


Fig. 4. NPC-SCR1 validated-v5 사전 배정 분모. 12 개 case, 2 회 반복, 3 개 arm 으로 72 개 row(arm 당 24 개), NPC-ENG1 16 개와 합쳐 88 개 (R 40 개, P 24 개, O 24 개) 다. P 는 실제 Codex typed intent, R 은 작성된 rule interpreter, O 는 same-model-family auxiliary reference 이며 independent-source row 는 0 이다. ENG-X01/SCR-X01 과 v1–v4 provenance 를 보존하고, case pool 밖 full-loop demo 는 포함하지 않는다. 이는 고정 사례 계수이지 recovery efficacy, oracle 비교, normal-play 또는 player benefit 이 아니다.

표와 그림의 수치는 고정 사례 계수일 뿐이다. packet 검증 전에는 목표 도달, 회복 평가 가능/불가능, completion/failure 를 성공률이나 효능으로 읽지 않으며, hash-bound receipt 가 없는 행을 결과표에 채우지 않는다.

보고 경계는 명시적이다. 동결 fixture 와 지정 검사 연산이 허용한 row 만 오프라인 표에 들어가고, 라이브 표는 자체 hash-bound packet 을 통해 도착하며 동결 오프라인 denominator 에는 들어가지 않는다. 각 lane 이 측정하지 않은 항목은 표 IV가 나열한다.

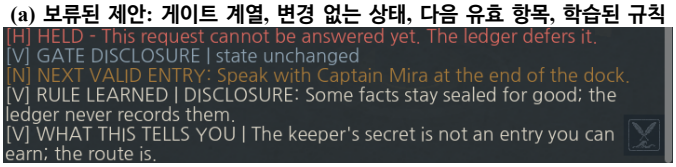
ENG1. Godot 4.7.1 의 두 번째 구현이 authored trace 를 replay 했다. 금지 disclosure 는 이전 hash 에서 보류되고 authorized commit 은 offline terminal hash 로 전진했다. 그림 5의 readout 과 근거는 pixel 이 아닌 trace 와 deterministic evaluation 이다. case-file board, hint ladder, coach card, chain prompt 는 committed fact 를 읽고, fail-closed dashboard 는 checks, holds, unreached in-scope checks 와 out-of-scope families 를 보여 준다. Fixture 는 core 52/52 와 extension 4 개이며 live model/Python transport/participant input/narrative result 가 없고 NPC-SCR1, normal-play/player study, usability/efficacy 근거가 아니다.

VII. 논의

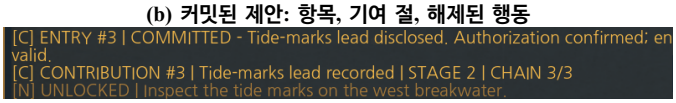
A. 확립 가능한 범위

동결 pilot 은 mechanism behavior 만 확립한다. encoded check 의 state isolation, 선언된 repair/fallback 경로, 명명된 corruption 거부와 row classification 을 확인하며 안정적인 detector 귀속은 하지 않는다. NPC-ENG1 은 synthetic closed-world evidence, NPC-SCR1 은 one-step screening 이며 normal-play episode 는 별도 illustrative artifact 다. 발화/model output 은 fixed goal 을 바꿀 수 없다. Python 은 typed intent 와 bounded public BFS 를 내고 role gate 는 player/Mira reveal

(a) 보류된 제안: 게이트 계열, 변경 없는 상태, 다음 유효 항목, 학습된 규칙



(b) 커밋된 제안: 항목, 기여 절, 해제된 행동



(c) 세 번의 커밋 뒤 엔드카드 영수증

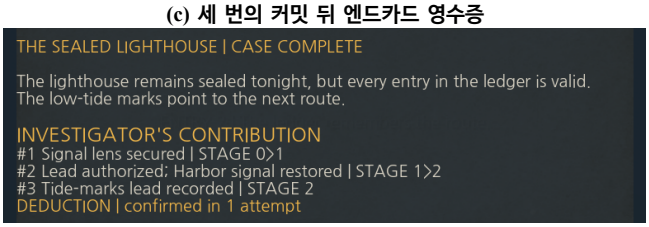


Fig. 5. 플레이어가 Godot slice 에서 읽는 commit gate. 작성된 fixture 하나의 scripted presentation stage 세 개에서 잘라낸 ledger(엔지니어링 작업 캡처, 1280×720; 예시일 뿐 근거가 아니다). (a) 금지된 disclosure 가 보류된다. ledger 는 게이트 계열을 이름 짓고, canonical state 가 변경되지 않았음을 밝히며, 다음 유효 항목을 가리키고, 보류를 플레이어가 재사용할 수 있는 규칙으로 바꾼다. (b) 유효한 제안이 커밋된다. 항목, committed snapshot 의 차분만으로 파생된 기여 절 (추가된 fact, 퀘스트 단계 1>2, case chain 2/3), 그리고 그것이 해제하는 행동이 이어진다. (c) 엔드카드는 기여 절 세 개, 보류 횟수, 그리고 리플레이가 보고하는 종단 state hash(`4b2310…`) 를 담은 validator 영수증을 나열한다. 모든 줄은 committed snapshot 의 presentation-only 판독이며 (그림 1 아래 strip), oracle label, 숨겨진 fact, live model, participant 는 개입하지 않는다. 패널은 usability, immersion, efficacy 어느 주장도 뒷받침하지 않는다.

을 전달하며 replan 은 observation 을 사용한다. P/R/O 는 adapter/control/auxiliary 이고 independent row 는 0 이므로 count 는 efficacy, superiority, generalization, human benefit, cost equality, semantic gold 또는 accuracy ceiling 을 추정하지 않는다.

Offline state-reading 은 ρ 가 전환할 수 없는 oracle-only 사례를 처리하고, typed error 만 쓰는 ρ 는 guided-repairable class 에서 oracle 과 일치한다. Live 에서 blind retry 와의 분리는 그 regime 에서만 나타났으며 already-valid/irreparable null, unpinned revision 과 quasi-replicated calls 는 sample-efficiency/model-capability 주장을 막는다. Self-Refine, Reflexion, Self-Debugging 이 선례지만 [48], [55], [56], ρ 는 deterministic pre-execution error 만 소비하고 실패 arm 을 격리하며 semantic repair 가 아니라 counterexample-guided synthesis 와 유사할 뿐이다 [57].

B. 선행연구와의 관계

TRACE-RPG 은 constrained decoder [5], [6], [7], 검색 및 메모리 구성요소 [8], [9], [10], 실행 가능한 환경과 benchmark [1], [2], [22], [25] 를 대체하지 않고 보완한다. Process-linked trace 는 AgentBoard 의 측정 관심사를 구현하지만 [23] 우월성의 근거는 아니다. 확증 protocol 은 아직 실행하지 않았다. 독립 semantic oracle 과 실제 blind-retry comparator 를 갖춘 사전등록 다중 모델 비교, validity 를 맞춘 retrieval/memory ablation, 정당화된 검정력과 mixed-effects 분석을 갖춘 blinded matched-validity 인간 연구가 필요하며, 현재 screening pilot 을 대체하지 않는다 [40], [37], [45].

VIII. 타당성 위협, 윤리, 산출물 범위

근거의 경계는 아홉 가지다. **L1:** encoded predicate 는 structured field 밖 의미와 행동 일관성을 보지 못하며 서사 품질은 미측정이다 [58]. **L2:** authored world 와 constructed live-state variant 하나로 cross-world, cross-policy, scale 일반화를 할 수 없다. **L3:** state-reading callback 은 oracle 이며 offline class 와 regime-dependent screening 은 일반 repair 품질이나 sample efficiency 를 확립하지 않는다. **L4:** hosted proposer 하나는 revision/token/sampling seed 를 고정하지 않으며, E2 screening 은 Python–Godot authorization round trip 을 측정하거나 benchmark 로 확립하지 않는다. **L5:** participant 와 affect data 가 없으므로 packet 은 engine-local 적합성과 measurement-ready protocol 일 뿐 comprehension, agency, enjoyment, immersion, player benefit 을 지지하지 않는다. **L6:** unkeyed hash 는 integrity 만 보장하고 writer authentication 이나 adversarial tamper resistance 는 제공하지 않는다. **L7:** single-writer 를 가정하며 concurrent-writer safety 는 미검증이다. **L8:** startup-dominated Godot fixture 하나의 16.7 ms 초과 sample 이라 portability 와 real-time 결과가 아니다. **L9:** 선택된 fixture 와 screening cell 은 population, safety-rate, model-ranking 추론을 지지하지 않는다. Internal validity 는 독립 policy expectation 과 fixture leakage 부재에, reproducibility 는 frozen software 와 artifact 에 의존한다. NPC 경로의 asynchronous deadline, duplicate/late-drop, session/epoch/goal/policy/revision/hash freshness gate 는 latency 나 quality benefit 가 아닌 protocol invariant 다. Live 취소는 local HTTP/directive invalidation 만 수행하므로 provider 가 deadline 까지 끝내거나 server bounded trace/cache 를 남길 수 있다. 반대로 `Core.cancel` 의 no-publication 보장은 unit/controller-only 이며 live route 취소 보장이 아니다. final predicate 와 player-request 단계는 game contract 에 고정되어 있고 human study, cross-world validation, independent semantic gold label, oracle 또는 recovery efficacy 증거는 없다. Core snapshot replay 는 redacted recorded-intent 를 epoch 별로 모델 호출 없이 재생하고 native replay 는 불변 입력을 `--npc-experiment/engine_fixture_reexecution` 으로 재실행하며 `--npc-replay` 명령은 사용하지 않는다.

현재 pilot 은 participant 또는 personal telemetry 를 처리하지 않는다. Human/affect study 와 LLM/crowd judge 는 아직 실행하지 않은 planned work 이며, 향후 review, informed consent, compensation disclosure, minimization, retention/deletion rule, opt-out, judge calibration 과 bias control 이 필요하다 [40], [41], [42], [43].

IX. 결론

TRACE-RPG 은 mutation 전에 payload 를 gate 하고 linked record/replay 로 outcome 을 감사한다. Frozen fixture 는 conformance 를, live screening 은 constructed repairable regime 에서만 ρ 와 blind retry 를 5/5 대 0/5 로 분리하는 pilot count 를 제공한다. Godot readout 은 committed-snapshot rule/contribution 을 드러내지만 cross-model/player/retrieval/memory/affect/commercial benefit 은 미검증이다. NPC 는 implemented fixed-goal prototype 이다. Python 은 typed intent/public-projection BFS 를 내고 Godot 만 canonical change 를 허가한다. State movement, cumulative budgets, observation-only replanning 과 zero-call replay 는 contract behavior 이지 efficacy 나 player-benefit 주장이 아니다.

AI 사용 고지

Large language model 은 prose, code 와 test drafting, citation-identity check 를 보조했고, hosted model 은 별도 표기된 live-screening candidate 를 생성했다. 모든 보고 count 는 deterministic runner 또는 hash-bound receipt 에서 온다. playable slice 의 UI 아트 (초상, 인장, 패널 텍스처, case-file board, evidence icon, guide vignette) 와 prop mesh 하나 (evidence desk) 는 파일별 provenance 를 갖춘 curated AI 생성 asset 이며, 그림 5의 모든 상태 값, hash, ledger 줄은 committed snapshot 에서 엔진이 렌더링한 것이다. 저자는 claim 을 source artifact 에 대조했고 내용에 대한 책임을 진다.

데이터 및 코드 가용성

Repository 에는 implementation, build source, 동결된 offline 및 live packet, 결정론적 평가를 갖춘 playable Godot slice, 그리고 표 I과 IV를 생성하는 기계 검증 기여 · 참조 주제 · 실험 매트릭스가 들어 있다. Reviewer archive 나 DOI deposit 은 주장하지 않는다. Additive NPC 표와 그림은 NPC-GOAL-RECOVERY packet 에서 별도로 생성하며, 모든 assigned row 를 보존하고 project/protocol/schema hash 를 결합한다. token/cost/model-revision 을 사용할 수 없을 때는 0 이 아니라 null 로 기록한다.

References

[1] M.-A. Côté, Á. Kádár, X. Yuan *et al.*, "TextWorld: A learning environment for text-based games," in *Computer Games*, ser. Communications in Computer and Information Science. Springer International Publishing, 2019, vol. 1017, pp. 41–75.

[2] R. Wang, P. Jansen, M.-A. Côté, and P. Ammanabrolu, "ScienceWorld: Is your agent smarter than a 5th grader?" in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2022, pp. 11 279–11 298.

[3] N. Weir, R. Thomas, R. d'Amore, K. Hill, B. Van Durme, and H. Jhamtani, "Ontologically faithful generation of non-player character dialogues," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 9212–9242.

[4] T. Ashby, B. K. Webb, G. Knapp, J. Searle, and N. Fulda, "Personalized quest and dialogue generation in role-playing games: A knowledge graph- and language model-based approach," in *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, 2023, pp. 1–20.

[5] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 10 932–10 952.

[6] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Prompting is programming: A query language for large language models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 1946–1969, 2023.

[7] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 9895–9901.

[8] X. He, Y. Tian, Y. Sun *et al.*, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 132 876–132 907.

[9] B. Jiménez Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 59 532–59 569.

[10] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," in *ECAI 2025*, ser. Frontiers in Artificial Intelligence and Applications, vol. 413. IOS Press, 2025, pp. 2993–3000.

[11] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, 2023, pp. 1–22.

[12] Y. Shao, L. Li, J. Dai, and X. Qiu, "Character-LLM: A trainable agent for role-playing," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, pp. 13 153–13 187.

[13] N. Akoury, Q. Yang, and M. Iyyer, "A framework for exploring player perceptions of LLM-generated dialogue in commercial video games," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 2295–2311.

[14] M. Hochreiter, S. Krigl stein, and G. Wallner, "Beyond pre-defined scripts: Player perceptions on generative non-player character dialogues," in *Proceedings of the 31st International Conference on Intelligent User Interfaces*. ACM, 2026, pp. 2004–2018.

[15] M. Yin, H. Zu, W. Gu *et al.*, "How contextualized generative AI shapes player experience in games," *Entertainment Computing*, vol. 58, p. 101194, 2026.

[16] V. Figueiredo and D. Elumeze, "Symbolically scaffolded play: Designing role-sensitive prompts for generative NPC dialogue," 2025, preprint.

[17] G. N. Yannakakis and J. Togelius, "Experience-driven procedural content generation," *IEEE Transactions on Affective Computing*, vol. 2, no. 3, pp. 147–161, 2011.

[18] D. Melhart, D. Gravina, and G. N. Yannakakis, "Moment-to-moment engagement prediction through the eyes of the observer: PUBG streaming on Twitch," in *FDG '20: Proceedings of the 15th International Conference on the Foundations of Digital Games*. ACM, 2020, pp. 1–10.

[19] Z. Wang, Q. Guo, R. Singh, and X. Hu, "Do vision language models understand human engagement in games?" 2026, preprint.

[20] L. Fan, G. Wang, Y. Jiang *et al.*, "MineDojo: Building open-ended embodied agents with internet-scale knowledge," in *Advances in Neural Information Processing Systems*, vol. 35. Neural Information Processing Systems Foundation, 2022, pp. 18 343–18 362.

[21] D. Pérez-Liébana, J. Liu, A. Khalifa, R. D. Gaina, J. Togelius, and S. M. Lucas, "General video game AI: A multitrack framework for evaluating agents, games, and content generation algorithms," *IEEE Transactions on Games*, vol. 11, no. 3, pp. 195–214, 2019.

[22] X. Liu, H. Yu, H. Zhang *et al.*, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, 2024, pp. 52 989–53 046.

[23] C. Ma, J. Zhang, Z. Zhu *et al.*, "AgentBoard: An analytical evaluation board of multi-turn LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37. Neural Information Processing Systems Foundation, 2024, pp. 74 325–74 362.

[24] X. Zhou, H. Zhu, L. Mathur *et al.*, "SOTOPIA: Interactive evaluation for social intelligence in language agents," in *International Conference on Learning Representations*, 2024, pp. 40 975–41 019.

[25] D. Paglieri, B. Cupiał, S. Coward *et al.*, "BALROG: Benchmarking agentic LLM and VLM reasoning on games," in *International Conference on Learning Representations*, 2025, pp. 96 666–96 702.

[26] P. Le Pelletier de Woillemont, R. Labory, and V. Corruble, "Automated play-testing through RL based human-like play-styles generation," *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 18, no. 1, pp. 146–154, 2022.

[27] Y. Ma, J. Yan, B. Shi *et al.*, "Can LLM agents stick to the script? a benchmark for long-horizon consistency in interactive narratives," 2026, preprint (v1); ICML 2026 acceptance comment on arXiv, proceedings record unverified.

[28] S. Kambhampati, K. Valmeekam, L. Guan *et al.*, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 2024, pp. 22 895–22 907.

[29] B. Ichter, A. Brohan, Y. Chebotar *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," in *Proceedings of the 6th Conference on Robot Learning (CoRL 2022)*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318.

[30] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025, preprint (v3); ICSE 2026 acceptance stated on arXiv, proceedings record unverified.

[31] Z. Zeng, S. Li, J. Xi, A. Zhu, and P. Ammanabrolu, "Setting the DC: Tool-grounded D&D simulations to test LLM agents," in *NeurIPS 2025 Workshop on Generative and Protective AI for Content Creation (GenProCC)*, 2025, non-archival workshop paper; OpenReview/PDF list Zeng first, with Zeng and Li as equal contributors; NeurIPS/GenProCC lists display Li first.

[32] E. Zhou, S. Basavatia, M. Siam, Z. Chen, and M. O. Riedl, "STORY2GAME: Generating (almost) everything in an interactive fiction game," 2025, preprint.

[33] S. Buongiorno, L. Klinkert, Z. Zhuang, T. Chawla, and C. Clark, "PANGeA: Procedural artificial narrative using generative AI for turn-based, role-playing video games," *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, vol. 20, no. 1, pp. 156–166, 2024.

[34] M. Vaucher, S. Silveira, S. Góngora, and L. Chiruzzo, "IVIE: A neuro-symbolic approach to incremental and validated generation of interactive fiction worlds," in *Proceedings of the 17th International Conference on Computational Creativity (ICCC'26)*. Association for Computational Creativity, 2026, archival proceedings (ISBN); preprint arXiv:2606.13348.

[35] S. Góngora, L. Chiruzzo, G. Méndez, and P. Gervás, "World-state transformations for neuro-symbolic interactive storytelling," 2026, preprint.

[36] S. Geng, H. Cooper, M. Moskal *et al.*, "JSONSchemaBench: A rigorous benchmark of structured outputs for language models," 2025, preprint (v3).

[37] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, "Deep reinforcement learning at the edge of the statistical precipice," in *Advances in Neural Information Processing Systems*, vol. 34. Curran Associates, Inc., 2021, pp. 29 304–29 320.

[38] J. Pineau, P. Vincent-Lamarre, K. Sinha *et al.*, "Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

[39] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *Journal of Machine Learning Research*, vol. 21, no. 248, pp. 1–43, 2020.

[40] C. van der Lee, A. Gatt, E. van Miltenburg, S. Wubben, and E. Krahmer, "Best practices for the human evaluation of automatically generated text," in *Proceedings of the 12th International Conference on Natural Language Generation*. Association for Computational Linguistics, 2019, pp. 355–368.

[41] M. Karpinska, N. Akoury, and M. Iyyer, "The perils of using Mechanical Turk to evaluate open-ended text generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021, pp. 1265–1285.

[42] L. Zheng, W.-L. Chiang, Y. Sheng *et al.*, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36. Neural Information Processing Systems Foundation, 2023, pp. 46 595–46 623.

[43] G. H. Chen, S. Chen, Z. Liu, F. Jiang, and B. Wang, "Humans or LLMs as the judge? a study on judgement bias," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 8301–8327.

[44] D. Lakens, A. M. Scheel, and P. M. Isager, "Equivalence testing for psychological research: A tutorial," *Advances in Methods and Practices in Psychological Science*, vol. 1, no. 2, pp. 259–269, 2018.

[45] D. J. Barr, R. Levy, C. Scheepers, and H. J. Tily, "Random effects structure for confirmatory hypothesis testing: Keep it maximal," *Journal of Memory and Language*, vol. 68, no. 3, pp. 255–278, 2013.

[46] E. Feng, W. Zhou, Z. Liu *et al.*, "Get experience from practice: LLM agents with record & replay," 2025, preprint.

[47] Y. Sun, H. Wang, D. Li, G. Wang, and H. Zhang, "The emperor's new clothes in benchmarking? a rigorous examination of mitigation strategies for LLM benchmark data contamination," 2025, preprint (v1).

[48] A. Madaan, N. Tandon, P. Gupta *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 46 534–46 594.

[49] M. O. Riedl, C. J. Saretto, and R. M. Young, "Managing interaction between users and agents in a multi-agent storytelling environment," in *Proceedings of the Second International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2003, pp. 741–748.

[50] R. E. Fikes and N. J. Nilsson, "STRIPS: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.

[51] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, 2000.

[52] T. Haerder and A. Reuter, "Principles of transaction-oriented database recovery," *ACM Computing Surveys*, vol. 15, no. 4, pp. 287–317, 1983.

[53] R. Evans and E. Short, "Versu—a simulationist storytelling system," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 6, no. 2, pp. 113–130, 2014, predecessor title of IEEE Transactions on Games.

[54] S. G. Ware and C. Siler, "Sabre: A narrative planner supporting intention and deep theory of mind," in *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, vol. 17, no. 1, 2021, pp. 99–106.

[55] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023, pp. 8634–8652.

[56] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024, accepted poster; OpenReview archival record; preprint arXiv:2304.05128.

[57] A. Solar-Lezama, L. Tancau, R. Bodík, S. Seshia, and V. Saraswat, "Combinatorial sketching for finite programs," in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2006, pp. 404–415.

[58] Y. Huang, G. Chen, J. Yao *et al.*, "Beyond state consistency: Behavior consistency in text-based world models," 2026, preprint.